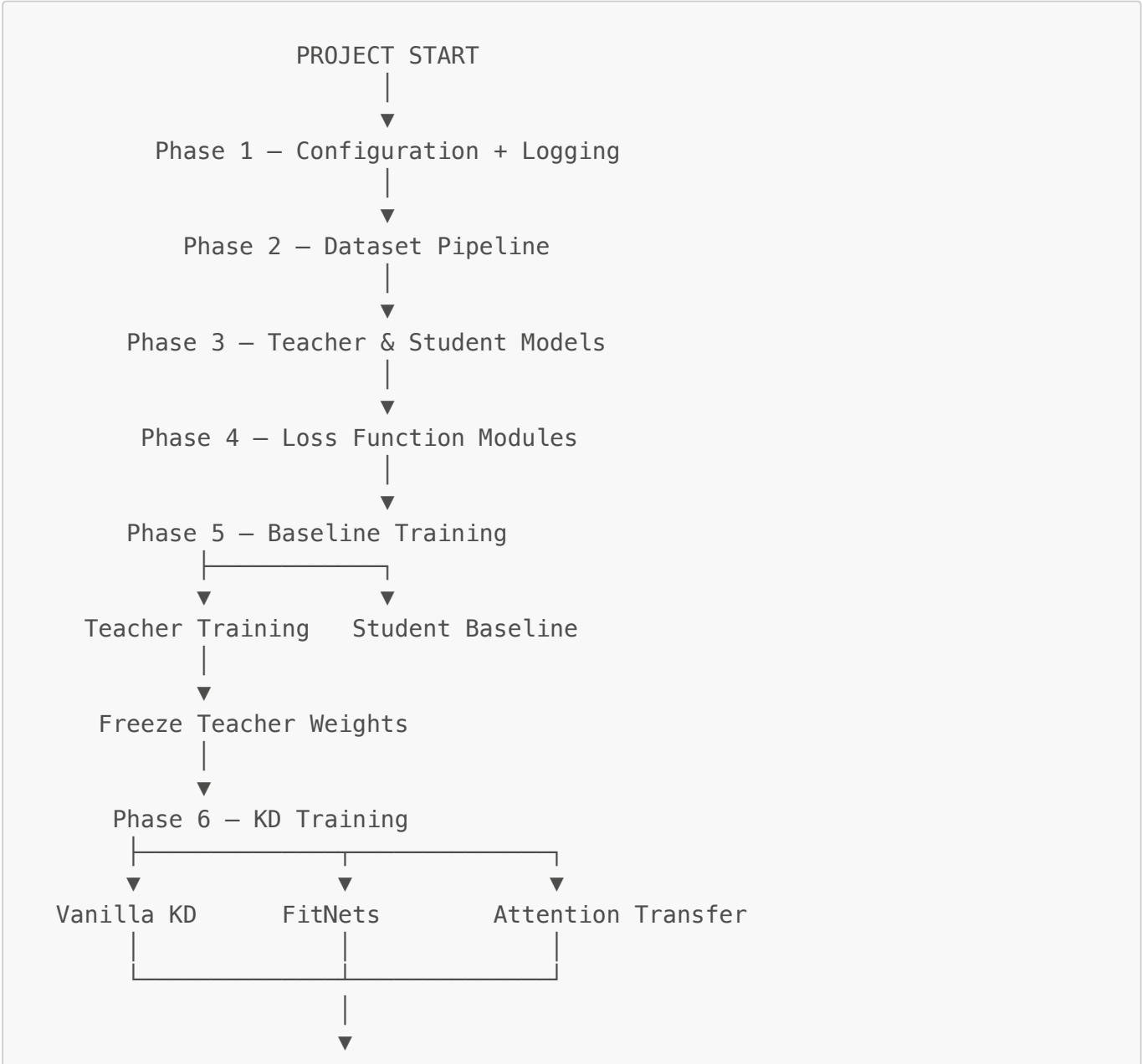
Master Project Execution & Implementation Plan

Comparative Study of Knowledge Distillation Algorithms (PyTorch)

This plan is designed as a **research-grade execution roadmap**, where every phase builds on the previous one. The guiding principles are:

- **Modularity** — each component has a single responsibility.
- **Reproducibility** — experiments should be rerunnable with only a config change.
- **Extensibility** — adding another KD algorithm later should require minimal changes.
- **Fair Comparison** — every experiment uses the exact same training pipeline except for the KD method.

Overall Workflow



```
       Phase 7 — Evaluation & Profiling
                      │
                      ▼
           Comparison Tables & Plots
                      │
                      ▼
                 Final Report
```

Dependency Order

```
Configs
    ↓
Logger + Seed
    ↓
Dataset Pipeline
    ↓
Models
    ↓
Feature Hooks
    ↓
Losses
    ↓
Teacher Training
    ↓
Student Baseline
    ↓
Freeze Teacher
    ↓
KD Training
    ↓
Evaluation
```

Why this order matters

- Configs must exist before any script can load parameters.
- Logging must exist before training starts to ensure experiments are reproducible.
- Dataset pipeline must be stable before debugging models.
- Models must exist before feature hooks.
- Feature hooks are required before FitNets/AT.
- Losses require hooks and models.
- Teacher must be trained before any KD.
- Student baseline provides the comparison point.
- KD methods reuse the same teacher.

Phase 1 — Configuration & Logging Foundations

1.1 Configuration Architecture

Files

```
configs/
    cifar10.yaml
    cifar100.yaml
    kd_config.yaml
```

Responsibilities

Centralize every hyperparameter.

Examples

- dataset
- batch size
- epochs
- optimizer
- scheduler
- checkpoint paths
- KD parameters
- logging frequency

Nothing inside trainers should hardcode values.

Checklist

- [ ] Create CIFAR-10 config
- [ ] Create CIFAR-100 config
- [ ] Create KD config
- [ ] Add experiment name
- [ ] Add checkpoint paths
- [ ] Add optimizer settings
- [ ] Add scheduler settings
- [ ] Add random seed
- [ ] Add device selection
- [ ] Validate YAML loading

1.2 Random Seed Utility

File

```
utils/seed.py
```

Responsibilities

- NumPy seed
- PyTorch seed
- CUDA seed
- deterministic mode

Checklist

- [ ] Seed Python
- [ ] Seed NumPy
- [ ] Seed Torch
- [ ] Seed CUDA
- [ ] Enable deterministic execution
- [ ] Verify identical runs

1.3 Logging Infrastructure

File

```
utils/logger.py
```

Responsibilities

- Console logging
- File logging
- Training history
- Metric storage
- Timestamping

Checklist

- [ ] Create experiment folder
- [ ] Save logs
- [ ] Save config copy
- [ ] Log epoch metrics
- [ ] Log learning rate
- [ ] Log elapsed time
- [ ] Log checkpoint events

Phase 2 — Data Pipeline & Augmentations

2.1 Transform Pipeline

Files

```
datasets/transforms.py
```

Responsibilities

Training transforms

Testing transforms

Normalization

Checklist

- [ ] Training transforms
- [ ] Test transforms
- [ ] CIFAR normalization
- [ ] Configurable augmentations
- [ ] Tensor conversion
- [ ] Unit test output size

2.2 Dataset Loader

File

```
datasets/cifar.py
```

Responsibilities

- Download dataset
- Build DataLoaders
- Return train/test loaders
- Support CIFAR10
- Support CIFAR100

Checklist

- [ ] CIFAR10 loader
- [ ] CIFAR100 loader

- [ ] Config integration
- [ ] DataLoader creation
- [ ] Worker support
- [ ] Pin memory
- [ ] Verify batch shape

Phase 3 — Model Architectures & Feature Extraction

3.1 Teacher Wrapper

File

```
models/resnet_teacher.py
```

Responsibilities

- Load ResNet34
- Modify classifier
- Load checkpoints
- Freeze weights

Checklist

- [ ] Build model
- [ ] Replace classifier
- [ ] Load weights
- [ ] Freeze model
- [ ] Verify output shape

3.2 Student Wrapper

File

```
models/resnet_student.py
```

Responsibilities

- Load ResNet18
- Replace classifier
- Train normally

Checklist

- [ ] Build model
- [ ] Replace classifier
- [ ] Verify forward pass

3.3 Feature Hook System

File

```
models/feature_hooks.py
```

Responsibilities

Collect intermediate activations.

Required by

- FitNets
- Attention Transfer

Checklist

- [ ] Register hooks
- [ ] Remove hooks safely
- [ ] Store outputs
- [ ] Multiple layer support
- [ ] Debug feature shapes

3.4 Regressor Module

File

```
models/regressor.py
```

Responsibilities

Feature dimension matching.

Used only in FitNets.

Checklist

- [ ] Build projection layer

- [ ] Support channel mismatch
- [ ] Verify output shape

Phase 4 — Loss Functions Engineering

4.1 Vanilla KD

File

```
losses/vanilla_kd.py
```

Responsibilities

- CE loss
- KL loss
- Temperature scaling
- Alpha weighting

Checklist

- [ ] CE implementation
- [ ] KL implementation
- [ ] Temperature support
- [ ] Alpha weighting
- [ ] Unit test

4.2 FitNet Loss

File

```
losses/fitnet.py
```

Responsibilities

- Hint loss
- Regressor integration
- Total loss

Checklist

- [ ] Hint loss

- [ ] Regressor support
- [ ] Total loss
- [ ] Shape validation

4.3 Attention Transfer

File

```
losses/attention_transfer.py
```

Responsibilities

- Generate attention maps
- Normalize maps
- Compute attention loss

Checklist

- [ ] Attention map generation
- [ ] Normalization
- [ ] Attention loss
- [ ] Total loss

4.4 Shared Cross Entropy

File

```
losses/cross_entropy.py
```

Responsibilities

Reusable CE wrapper.

Checklist

- [ ] Wrapper
- [ ] Label smoothing support
- [ ] Unit tests

Phase 5 — Baseline Model Training

5.1 Teacher Training

File

```
trainers/train_teacher.py
```

Responsibilities

Train ResNet34.

Outputs

- checkpoint
- logs
- metrics

Checklist

- [ ] Training loop
- [ ] Validation loop
- [ ] LR scheduler
- [ ] Checkpoint saving
- [ ] Best model saving
- [ ] Resume support

5.2 Student Baseline

File

```
trainers/train_baseline.py
```

Responsibilities

Train ResNet18 without KD.

Checklist

- [ ] Independent training
- [ ] Save metrics
- [ ] Save checkpoint
- [ ] Compare against teacher

Phase 6 — Knowledge Distillation Pipeline

6.1 Vanilla KD Trainer

File

```
trainers/train_vanilla.py
```

Pipeline

```
Teacher
      |
      ▼
 Teacher Logits
      |
      ▼
KD Loss
      |
      ▼
Student Update
```

Checklist

- [ ] Freeze teacher
- [ ] Forward teacher
- [ ] Forward student
- [ ] KD loss
- [ ] CE loss
- [ ] Backprop
- [ ] Save checkpoint

6.2 FitNets Trainer

Pipeline

```
Teacher Feature
        |
        ▼
     Regressor
        |
        ▼
Student Feature
        |
        ▼
Hint Loss
        |
```

```
        ▼
Student Update
```

Checklist

- [ ] Register hooks
- [ ] Build regressor
- [ ] Hint loss
- [ ] Stage-1 hint training (if implemented)
- [ ] Stage-2 KD fine-tuning
- [ ] Save checkpoints

6.3 Attention Transfer Trainer

Pipeline

```
Teacher Features
        │
        ▼
Attention Maps
        │
        ▼
Student Attention Maps
        │
        ▼
AT Loss
        │
        ▼
Student Update
```

Checklist

- [ ] Register hooks
- [ ] Generate attention maps
- [ ] Compute AT loss
- [ ] Total loss
- [ ] Save checkpoints

Phase 7 — Evaluation, Benchmarking & Comparative Profiling

7.1 Metrics

File

```
evaluation/metrics.py
```

Responsibilities

Compute:

- Top-1 Accuracy
- Top-5 Accuracy
- Loss
- Precision (optional if added later)

Checklist

- [ ] Top-1
- [ ] Top-5
- [ ] Average loss
- [ ] Validation utilities

7.2 Model Profiling

File

```
evaluation/profiler.py
```

Responsibilities

Measure:

- Parameters
- FLOPs
- Latency
- Model size
- GPU memory
- Training time

Checklist

- [ ] Parameter count
- [ ] FLOPs
- [ ] Latency benchmark

- [ ] Memory profiling
- [ ] Model size
- [ ] Compression ratio

7.3 Comparative Benchmarking

File

```
evaluation/compare.py
```

Responsibilities

Generate the final comparison across:

- Teacher
- Student Baseline
- Vanilla KD
- FitNets
- Attention Transfer

Produce:

- Markdown table
- CSV summary
- Publication-ready plots

Checklist

- [ ] Load all experiment logs
- [ ] Aggregate metrics
- [ ] Export CSV
- [ ] Generate comparison table
- [ ] Plot accuracy vs. efficiency
- [ ] Plot training curves
- [ ] Verify reproducibility across repeated runs (if available)

Final Deliverables

By the end of the project, the repository should include:

- [ ] Fully configurable experiment system
- [ ] Reproducible data pipeline
- [ ] Teacher (ResNet-34) implementation
- [ ] Student (ResNet-18) implementation
- [ ] Feature hook infrastructure

- [ ] Regressor module for FitNets
- [ ] Vanilla KD loss
- [ ] FitNets loss
- [ ] Attention Transfer loss
- [ ] Teacher training pipeline
- [ ] Student baseline training pipeline
- [ ] Vanilla KD trainer
- [ ] FitNets trainer
- [ ] Attention Transfer trainer
- [ ] Unified evaluation metrics
- [ ] Profiling utilities
- [ ] Comparative benchmarking and reporting
- [ ] Experiment logs, checkpoints, and reproducible configuration files

---

Ready to Start

Are you ready to begin Phase 1 (Configuration & Data Pipeline)? We'll start by implementing the configuration system (`configs/*.yaml`) together with the foundational utilities in `utils/logger.py` and `utils/seed.py`, then move into the dataset pipeline (`datasets/transforms.py` and `datasets/cifar.py`) before progressing to the models.